

# 601.433/633 Intro Algorithms

## Recitation 4

Fall 2026

### Data Structures

- (a) This problem is about the Union-Find algorithm which we analyzed to show that the amortized cost of an operation is  $O(\log^* n)$ . Suppose we do the following operations in order, where ties in rank of the Link operation are broken by increasing the rank of the first argument (as in class).

- Make-Set(a)
- Make-Set(b)
- Make-Set(c)
- Make-Set(d)
- Union(a,b)
- Union(c,d)
- Union(b,d)

Draw the resulting forest.

**Solution.** Recall that Make-Set( $x$ ) sets  $x \rightarrow \text{parent} = x$  and  $x \rightarrow \text{rank} = 0$ . Also, Union( $x,y$ ) = Link(Find( $x$ ), Find( $y$ )). If the two roots passed to Link have equal rank, then the second root becomes a child of the first root, and the rank of the first root increases by 1.

In the figures below, an arrow points from a node to its parent. No arrow is drawn when a node is a root.

#### Make-Set operations:

Each of the 4 Make-Set calls results in

$$a \rightarrow \text{parent} = a, \quad a \rightarrow \text{rank} = 0$$

$$b \rightarrow \text{parent} = b, \quad b \rightarrow \text{rank} = 0$$

$$c \rightarrow \text{parent} = c, \quad c \rightarrow \text{rank} = 0$$

$$d \rightarrow \text{parent} = d, \quad d \rightarrow \text{rank} = 0,$$

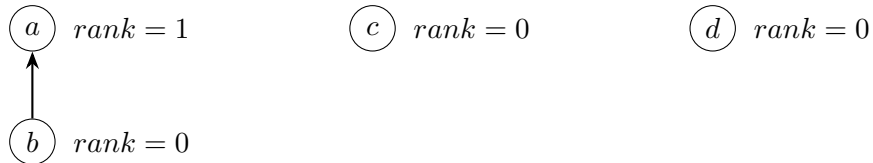
with the resulting forest being:

$\textcircled{a} \text{ rank} = 0$ 
 $\textcircled{b} \text{ rank} = 0$ 
 $\textcircled{c} \text{ rank} = 0$ 
 $\textcircled{d} \text{ rank} = 0$

**Union(a,b):**

We first compute  $\text{Find}(a)=a$  and  $\text{Find}(b)=b$ . Thus  $\text{Union}(a,b) = \text{Link}(a,b)$ . Since  $a$  and  $b$  both have rank 0, the tie is broken in favor of  $a$ . Hence  $b \rightarrow \text{parent} = a$  and  $a \rightarrow \text{rank} = 1$ .

The forest is



**Union(c,d):**

Using the exact same argument as above, we get  $d \rightarrow \text{parent} = c$  and  $c \rightarrow \text{rank} = 1$ , and the resulting forest is



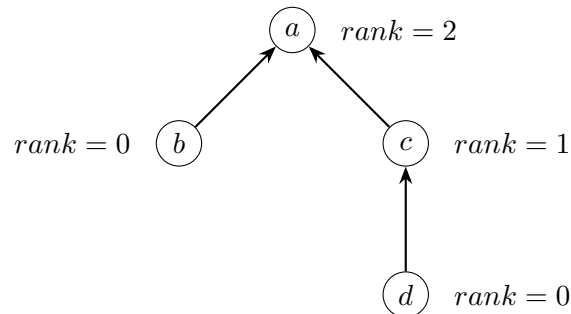
**Union(b,d):**

By definition,  $\text{Union}(b,d) = \text{Link}(\text{Find}(b), \text{Find}(d))$ . Since  $b \rightarrow \text{parent} = a$  and  $a$  is the root of the tree,  $\text{Find}(b)=a$ . Similarly,  $\text{Find}(d)=c$ . Both  $b$  and  $d$  already point directly to their respective roots, so these two Find operations do not change any parent pointers.

Hence,  $\text{Union}(b,d) = \text{Link}(a,c)$ .

Both  $a$  and  $c$  have rank 1. Since ties are broken in favor of the first argument,  $c \rightarrow \text{parent} = a$  and  $a \rightarrow \text{rank} = 2$ .

Thus, the final forest is



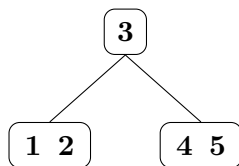
- (b) Suppose that we insert the keys 1,2,3,4,5,6 (in that order) into a B-tree with parameter  $t = 3$  using the insertion algorithm from class. Draw the resulting tree.

**Solution.** For a B-tree with parameter  $t = 3$ , maximum number of keys in a node  $= 2t - 1 = 5$ . So a node is full when it contains 5 keys. After inserting keys  $[1, 2, 3, 4, 5]$ , the root is still a leaf and has reached full capacity:

1 2 3 4 5

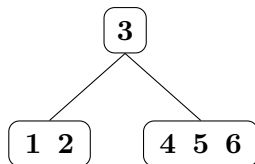
Before inserting 6, the root is full, so the insertion algorithm first splits it: The median key is 3, so it moves up to become the new root. The remaining keys are split into two children nodes, one with keys  $[1, 2]$  since they are less than the median and the other with keys  $[4, 5]$  since they are greater than the median.

Thus, immediately after the split, the tree is:



Since  $6 > 3$ , we descend into the right child and insert 6 into that leaf which now contains keys  $[4, 5, 6]$ .

So the final B-tree is:



# Hashing

We saw in class that universal hashing lets us give guarantees that hold for arbitrary (i.e. worst case) sets  $S$ , in expectation over our random choice of hash function. Let's work out some more of those guarantees.

- (a) Let  $H$  be a universal hash family from  $U$  to a table of size  $m$ . Let  $S \subseteq U$  be a set of  $m$  elements which we want to hash (so we're hashing the same number of elements as the size of the table). Prove that if we choose  $h$  from  $H$  uniformly at random, the expected number of pairs  $x, y \in S$  that collide is at most  $\frac{m-1}{2}$ .

**Solution.** This can be shown using very similar reasoning to Theorem 10.3.2 in the lecture notes. Let  $C_{xy} = 1$  if  $x$  and  $y$  collide, and 0 otherwise. Let  $C$  be the total number of collisions, so that

$$C = \sum_{x \in S} \sum_{\substack{y \in S \\ x < y}} C_{xy}$$

We know (from the definition of universal hashing) that  $\mathbf{E}[C_{xy}] \leq \frac{1}{m}$ . Thus, by linearity of expectation, we have that

$$\begin{aligned} \mathbf{E}[C] &= \sum_{x \in S} \sum_{\substack{y \in S \\ x < y}} \mathbf{E}[C_{xy}] \\ &\leq \frac{m(m-1)}{2} \cdot \frac{1}{m} \\ &= \frac{m-1}{2} \end{aligned}$$

- (b) Prove that with probability at least  $3/4$ , no bin in the table gets more than  $2\sqrt{m} + 1$  elements.

Hint: use part (a), and consider using Markov's Inequality. To remind you: if  $X$  is a nonnegative random variable with expectation  $\mathbf{E}[X]$ , then  $\mathbf{Pr}[X > k\mathbf{E}[X]] < 1/k$  for any  $k > 0$ . For example, the probability that  $X$  is more than 100 times its expectation is less than  $1/100$ .

**Solution.** We would like to compute  $\mathbf{Pr}[\text{no bin in the table gets more than } 2\sqrt{m} + 1 \text{ elements}]$ . This is equal to  $(1 - \mathbf{Pr}[\text{some bin in the table gets more than } 2\sqrt{m} + 1 \text{ elements}])$ .

Observe that, whenever one bin gets more than  $2\sqrt{m} + 1$  elements, there will be at least  $\binom{2\sqrt{m}+1}{2}$  collisions (one for every pair of elements in that bin). If we let  $C$  denote the total number of collisions again, we get that  $C \geq \binom{2\sqrt{m}+1}{2}$ . It follows that:

$$\mathbf{Pr}[\text{some bin in the table gets more than } 2\sqrt{m} + 1 \text{ elements}] \leq \mathbf{Pr}\left[C > \binom{2\sqrt{m}+1}{2}\right]$$

since if you think about the sample space, the event where some bin in the table gets more than  $2\sqrt{m} + 1$  elements is a subset of the event where at least  $\binom{2\sqrt{m}+1}{2}$  collisions take place.

Now a simple calculation reveals that

$$\binom{2\sqrt{m}+1}{2} = \frac{(2\sqrt{m}+1)2\sqrt{m}}{2} = 2m + \sqrt{m} > 4\frac{m-1}{2}.$$

Now note that by part (a),  $\mathbf{E}[C] \leq \frac{m-1}{2}$ . Putting this together and using Markov's inequality with  $k = 4$ , we get that

$$\begin{aligned} & \mathbf{Pr}[\text{some bin in the table gets more than } 2\sqrt{m} + 1 \text{ elements}] \\ & \leq \mathbf{Pr}\left[C > \binom{2\sqrt{m}+1}{2}\right] \\ & \leq \mathbf{Pr}\left[C > 4\frac{m-1}{2}\right] \\ & \leq \mathbf{Pr}[C > 4\mathbf{E}[C]] \\ & \leq 1/4. \end{aligned}$$

Thus with probability at least  $3/4$ , *no* bin in the table gets more than  $2\sqrt{m} + 1$  elements.